

Università degli Studi di Catania

Dipartimento di Ingegneria Elettrica, Elettronica e Informatica

Corso di Laurea Magistrale in Ingegneria Informatica

Gestore di Ordinazioni per Fast Food

Documentazione di Progetto

Ingegneria del Software A-Z

Studente:

Scala Salvatore Gaetano

Docente:

Prof. Orazio Tomarchio

Anno Accademico 2025/2026

Prefazione

Il seguente documento presenta una descrizione relativa allo sviluppo dell'applicazione “Gestore di Ordinazioni per Fast Food”, implementata durante il corso di Ingegneria del Software. Il software è stato sviluppato nel linguaggio Java, mentre la progettazione in UML è stata realizzata con il supporto di software dedicati.

Verranno presentate le versioni finali di ciascun elaborato ottenute al termine di tutte le fasi di progettazione iterativa, partendo dall'analisi dei requisiti fino alla modellazione orientata agli oggetti. Infine, vengono descritte le fasi conclusive di testing e di refactoring del programma, attraverso le quali sono state apportate migliorie all'architettura e verificata la correttezza della logica di business e delle interfacce.

Indice

Prefazione	1
1 Ideazione e analisi dei requisiti	4
1.1 Introduzione	4
1.2 Requisiti	4
1.3 Obiettivi e casi d'uso	5
1.4 Modello dei casi d'uso	6
1.5 Documento di Visione	11
1.6 Regole di business	11
1.7 Specifiche Supplementari	12
1.8 Glossario	12
2 Analisi Orientata agli Oggetti - Iterazione 1	13
2.1 Introduzione	13
2.2 Analisi Orientata agli Oggetti	13
2.2.1 Modello di Dominio	13
2.2.2 Diagramma di Sequenza di Sistema (SSD)	14
2.2.3 Contratti delle operazioni	15
2.3 Progettazione Orientata agli Oggetti	16
2.3.1 Diagrammi di Interazione (Sequenza)	17
2.3.2 Diagramma delle Classi di Progetto (DCD)	17
3 Analisi Orientata agli Oggetti - Iterazione 2	19
3.1 Introduzione	19
3.2 Analisi Orientata agli Oggetti	19
3.2.1 Modello di Dominio Aggiornato	19
3.2.2 Diagramma di Sequenza di Sistema (SSD)	20
3.2.3 Contratti delle operazioni	21
3.3 Progettazione Orientata agli Oggetti	21
3.3.1 Diagrammi di Interazione (Sequenza)	22
3.3.2 Diagramma delle Classi di Progetto (DCD) Aggiornato	22
4 Analisi Orientata agli Oggetti - Iterazione 3	24
4.1 Introduzione	24
4.2 Analisi Orientata agli Oggetti	24
4.2.1 Modello di Dominio Aggiornato	24
4.2.2 Diagramma di Sequenza di Sistema (SSD)	24

4.2.3	Contratti delle operazioni	25
4.3	Progettazione Orientata agli Oggetti	26
4.3.1	Diagrammi di Interazione (Sequenza)	26
4.3.2	Diagramma delle Classi di Progetto (DCD) Aggiornato	27
5	Testing	28
5.1	Introduzione	28
5.2	Individuazione dei casi di test e Testing Unitario (JUnit)	28
5.3	Test di Sistema	29
6	Architettura, Refactoring e Conclusioni	30
6.1	Architettura Model-View-Controller (MVC) e Struttura del Codice	30
6.2	Refactoring	32
6.3	Test di Accettazione e Conclusioni	32

Capitolo 1

Ideazione e analisi dei requisiti

1.1 Introduzione

Lo scopo della fase di ideazione è quello di effettuare un'indagine col fine di ricavare un'idea generale del progetto da realizzare e quindi ottenere delle stime di fattibilità tenendo conto dei tempi e delle risorse necessari. Per capire al meglio come sviluppare e comprendere i requisiti del progetto, nella fase di ideazione sono stati presi in considerazione i seguenti documenti: Modello dei Casi d'Uso, Documento di Visione, Specifiche Supplementari, Regole di Business e Glossario.

1.2 Requisiti

Il titolare di un fast food richiede la realizzazione di un software che permetta di automatizzare e migliorare la gestione delle ordinazioni da parte dei clienti. Il software deve rappresentare uno strumento integrato che segua il processo di ordinazione in tutte le sue fasi, a partire dalla composizione del menu fino alla consegna del vassoio. In particolare:

- Il cliente deve poter consultare un catalogo digitale (Menu) contenente tutti i prodotti acquistabili presso il fast food, suddivisi per categorie (es. Panini, Bibite, Fritti, Dessert).
- Il cliente deve poter inserire la quantità desiderata per ogni prodotto e personalizzare le proprie scelte (es. rimuovere cipolla, aggiungere bacon, variare la taglia); il sistema dovrà calcolare dinamicamente le variazioni di prezzo.
- Il cliente deve poter gestire un carrello virtuale, potendo modificare le quantità o rimuovere articoli prima del pagamento.
- La vendita si conclude al chiosco digitale con un pagamento elettronico simulato. In caso di successo, il sistema genera un numero di ordine univoco e stampa una ricevuta.
- Il personale di cucina deve avere a disposizione un terminale per visualizzare in tempo reale la coda degli ordini pagati e in attesa di preparazione.

- La cucina deve poter far avanzare lo stato dell'ordine (Ricevuto → In Preparazione → Pronto) e avere la possibilità di gestire più ordini in parallelo.
- Il sistema deve gestire l'aggiornamento di un monitor visibile al pubblico in sala per notificare ai clienti quando il loro ordine è pronto per il ritiro.
- Il manager del fast food deve poter gestire separatamente prodotti, ingredienti e categorie nel menu (inserire, modificare, eliminare) e configurare specifiche tipologie di promozioni (es. sconti percentuali, importi fissi, menu completi).
- Il manager del fast food deve poter generare dei report di vendita per analizzare l'incasso e il totale degli ordini evasi nella giornata.

1.3 Obiettivi e casi d'uso

Analizzando i requisiti riportati nel paragrafo precedente, sono stati individuati gli attori principali a cui è destinato il sistema e gli obiettivi che essi intendono portare a termine; da queste informazioni infine sono stati ricavati i casi d'uso principali.

Attore	Obiettivo	Caso d'uso
Cliente	Comporre un'ordinazione, impostare quantità e personalizzazioni, gestire il carrello e completare il pagamento.	UC1: Gestisci Ordinazione
Cucina	Visualizzare gli ordini in entrata e aggiornare progressivamente (anche in parallelo) lo stato di preparazione.	UC2: Gestisci Coda Cucina
Manager	Inserire, rimuovere o modificare entità distinte come prodotti, categorie e ingredienti nel catalogo.	UC3: Gestisci Menu (CRUD)
Manager	Creare, modificare o rimuovere specifiche tipologie di promozioni.	UC4: Gestisci Promozioni (CRUD)
Manager	Monitorare l'andamento delle vendite, calcolando il numero di ordini evasi e l'incasso totale.	UC5: Genera Report Vendite

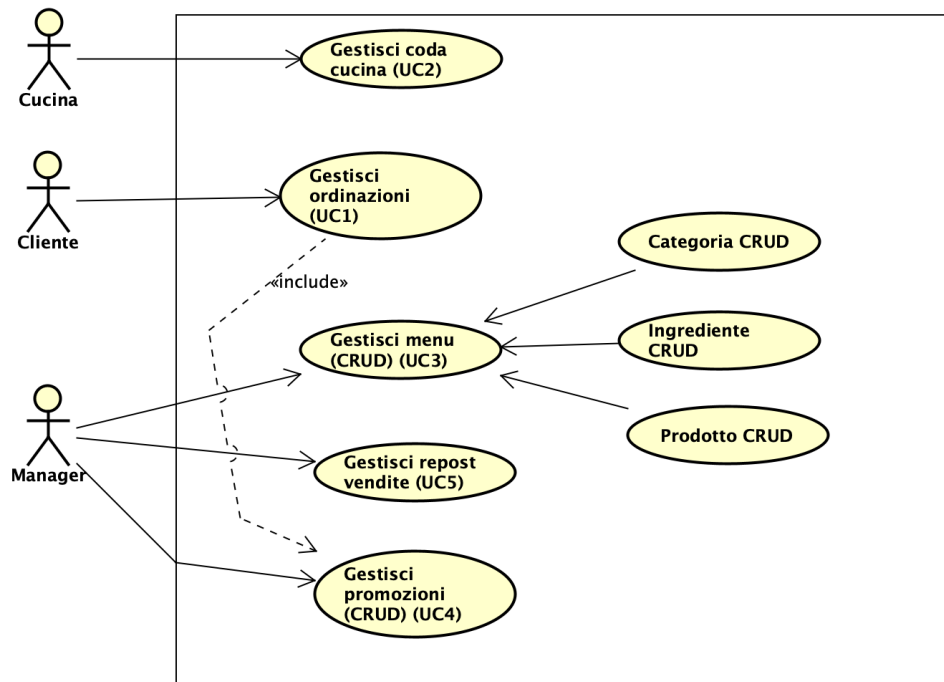


Figura 1.1: Obiettivi e casi d'uso in UML

1.4 Modello dei casi d'uso

Tra tutti i casi d'uso individuati, si è scelto di fornire una descrizione dettagliata per i casi d'uso principali del sistema, coprendo così i requisiti di dettaglio della fase di ideazione.

UC1: Gestisci Ordinazione

Nome del caso d'uso	UC1: Gestisci Ordinazione
Portata	Applicazione Kiosk Fast Food
Livello	Obiettivo utente
Attore primario	Cliente
Parti interessate e Interessi	Cliente: vuole comporre il proprio pasto in modo rapido, personalizzarlo senza errori e pagare velocemente. Proprietario: vuole che gli ordini siano precisi per evitare sprechi in cucina e massimizzare le vendite tramite up-selling.
Pre-condizioni	Il chiosco è operativo e connesso al database del menu e al sistema di pagamento.

Garanzia di successo	L'ordine viene salvato nel sistema, inviato alla coda della cucina, il pagamento è autorizzato e il cliente riceve lo scontrino con il numero di ritiro.
Scenario principale di successo	1. Il cliente tocca lo schermo per iniziare un nuovo ordine. 2. Il sistema mostra le categorie del menu. 3. Il cliente seleziona una categoria, sceglie un prodotto e ne inserisce la quantità desiderata. 4. Il sistema chiede se si desiderano personalizzazioni (intese come aggiunta di ingredienti extra, rimozione di ingredienti standard o variazioni di taglia/porzione). 5. Il cliente conferma il prodotto e le eventuali modifiche; il sistema lo aggiunge al carrello aggiornando il totale. <i>I passi da 2 a 5 vengono ripetuti finché il cliente lo desidera.</i> 6. Il cliente seleziona l'opzione "Termina e Paga". 7. Il sistema applica eventuali promozioni e mostra il totale finale. 8. Il cliente sceglie di pagare con carta e inserisce/avvicina la carta al POS (simulato). 9. Il sistema autorizza la transazione. 10. Il sistema invia l'ordine alla Cucina e stampa la ricevuta con il numero d'ordine.
Estensioni	*a. In qualsiasi momento, il sistema si riavvia improvvisamente: 1. Il carrello non salvato viene perso. 2. Il sistema torna alla schermata di benvenuto. 5a. Il cliente modifica la quantità, le personalizzazioni o rimuove un prodotto dal carrello: 1. Il sistema aggiorna i totali. 9a. Transazione rifiutata: 1. Il sistema notifica l'errore di pagamento. 2. Il cliente può riprovare con un'altra carta o annullare l'ordine.
Requisiti speciali	Non specificati.
Elenco varianti	Non specificati.
Frequenza	Legata all'affluenza dei clienti e agli acquisti che intendono effettuare.

UC2: Gestisci Coda Cucina

Nome del caso d'uso	UC2: Gestisci Coda Cucina
Portata	Applicazione Kiosk Fast Food (Lato Back-Office)

Livello	Obiettivo utente
Attore primario	Cucina (Personale)
Parti interessate e Interessi	Cucina: vuole una lista chiara e ordinata per gestire le preparazioni, potendo lavorare su più ordini contemporaneamente per massimizzare l'efficienza.
Pre-condizioni	Il terminale della cucina è autenticato e connesso al server centrale.
Garanzia di successo	Lo stato dell'ordine viene aggiornato nel database e le informazioni vengono riflesse sul monitor in sala per il cliente.
Scenario principale di successo	1. Il sistema mostra la coda degli ordini ricevuti in ordine cronologico. 2. Il personale di cucina seleziona uno o più ordini dalla lista (il sistema supporta la gestione e visualizzazione di più ordini in parallelo) per vederne i dettagli, le quantità e le personalizzazioni. 3. Il personale contrassegna gli ordini presi in carico come "In Preparazione". 4. Il sistema aggiorna il monitor clienti in sala spostando i numeri corrispondenti nella colonna "In Preparazione". 5. Una volta completato un vassoio, il personale contrassegna il relativo ordine come "Pronto". 6. Il sistema emette un avviso acustico in sala e sposta il numero nella colonna "Pronto". 7. Il cliente ritira l'ordine e il personale lo contrassegna come "Ritirato", archiviandolo.
Estensioni	*a. In qualsiasi momento, il sistema fallisce e ha un arresto improvviso: 1. Il personale riavvia il terminale. 2. Il sistema ricostruisce lo stato precedente della coda. 2a. Un ingrediente per una personalizzazione è esaurito: 1. Il personale di cucina seleziona l'ordine e lo contrassegna con un "Avviso al banco". 2. Il personale di sala concorda una sostituzione con il cliente.
Requisiti speciali	Non specificati.
Elenco varianti	Per ciascun ordine devono essere visualizzati: numero ordine, prodotti, quantità, personalizzazioni, stato.
Frequenza	Continua durante l'orario di apertura.
Problemi Aperti	Il sistema dovrebbe gestire l'annullamento di un ordine da parte della cucina in caso di problemi critici?

UC3: Gestisci Menu (Prodotti, Ingredienti e Categorie, CRUD)

Nome del caso d'uso	UC3: Gestisci Menu (Prodotti, Ingredienti e Categorie, CRUD)
Portata	Applicazione Kiosk Fast Food (Pannello Manager)
Livello	Obiettivo utente
Attore primario	Manager
Parti interessate e Interessi	Manager: vuole mantenere il catalogo aggiornato organizzando in modo separato e strutturato i Prodotti, gli Ingredienti (standard o extra) e le Categorie di visualizzazione.
Pre-condizioni	Il Manager è autenticato nel pannello di amministrazione.
Garanzia di successo	I database di prodotti, ingredienti o categorie vengono aggiornati correttamente e i chioschi in sala riflettono immediatamente le modifiche.
Scenario principale di successo (Inserimento)	1. Il manager richiede al sistema l'inserimento nel catalogo di una nuova entità (Prodotto, Ingrediente o Categoria). 2. Il sistema richiede al manager le informazioni specifiche dell'entità scelta. 3. Il manager inserisce i dati (es. per il prodotto: nome, prezzo base, ingredienti correlati, categoria di appartenenza) e conferma l'operazione.
Scenari alternativi	1a. Modifica di un'entità: 1. Il sistema richiede al manager i dati aggiornati. 2. Il manager aggiorna i campi di interesse e conferma. 1b. Ricerca di un'entità: 1. Il sistema richiede al manager di inserire le informazioni nei campi di ricerca. 2. Il manager inserisce le informazioni a sua disposizione. 3. Il sistema mostra l'elenco dei risultati che soddisfano i criteri. 1c. Eliminazione di un'entità: 1. Il sistema chiede al manager di confermare l'eliminazione. 2. Il manager conferma. 3. Il sistema rimuove o disattiva l'entità dal menu.

UC4: Gestisci Promozioni (CRUD)

Nome del caso d'uso	UC4: Gestisci Promozioni (CRUD)
Portata	Applicazione Kiosk Fast Food (Pannello Manager)
Livello	Obiettivo utente

Attore primario	Manager
Parti interessate e Interessi	Manager: vuole creare strategie di marketing impostando regole specifiche per sconti e promozioni.
Pre-condizioni	Il Manager è autenticato nel pannello di amministrazione.
Garanzia di successo	Il database delle promozioni viene aggiornato e il sistema applica le logiche corrette ai carrelli validi.
Scenario principale di successo (Inserimento)	1. Il manager richiede al sistema l'inserimento di una nuova promozione. 2. Il sistema richiede al manager le informazioni relative alla promozione. 3. Il manager seleziona la tipologia di promozione (es. Sconto Percentuale, Sconto a Importo Fisso, Formula Combo "Menu Completo", Offerta "Prendi 2 Paghi 1") e ne inserisce le regole di validità. 4. Il sistema chiede al manager il valore dello sconto da applicare in base al tipo scelto. 5. Il manager inserisce i dati, il periodo di validità e conferma.
Scenari alternativi	1a. Modifica di una promozione: 1. Il sistema richiede al manager i dati aggiornati. 2. Il manager aggiorna i campi di interesse e conferma. 1b. Ricerca di una promozione: 1. Il sistema richiede al manager di inserire le informazioni nei campi di ricerca. 2. Il manager inserisce le informazioni a sua disposizione. 3. Il sistema mostra l'elenco delle promozioni trovate. 1c. Eliminazione di una promozione: 1. Il sistema chiede al manager di confermare l'eliminazione della promozione. 2. Il manager conferma. 3. Il sistema disattiva ed elimina la promozione.

UC5: Genera Report Vendite

Nome del caso d'uso	UC5: Genera Report Vendite
Portata	Applicazione Kiosk Fast Food (Pannello Manager)
Livello	Obiettivo utente
Attore primario	Manager
Parti interessate e Interessi	Manager: vuole monitorare l'andamento delle vendite, conoscendo il numero esatto di ordini saldati e l'incasso totale della giornata per valutare l'efficienza del fast food.

Pre-condizioni	Il Manager è autenticato nel pannello di amministrazione. Esiste uno storico degli ordini nel sistema.
Garanzia di successo	Il sistema calcola accuratamente e mostra a schermo il numero totale di ordini ricevuti e la somma dei relativi incassi.
Scenario principale di successo	1. Il manager richiede al sistema la generazione del report delle vendite. 2. Il sistema interroga lo storico degli ordini saldati. 3. Il sistema calcola il numero totale di ordini e la somma matematica degli importi pagati. 4. Il sistema mostra a video il report riepilogativo.
Scenari alternativi	2a. Nessun ordine nello storico: 1. Il sistema notifica che non ci sono vendite da riportare.

1.5 Documento di Visione

Il sistema "Gestore di Ordinazioni per Fast Food" ha l'obiettivo di abbattere i tempi di attesa in coda alle casse fisiche, demandando la fase di scelta e pagamento al cliente tramite chioschi self-service. Questo permette di ottimizzare le risorse umane, concentrandole nella preparazione e nell'assistenza in sala, migliorando al contempo l'accuratezza degli ordini grazie all'eliminazione degli errori di comprensione vocale.

1.6 Regole di business

Per il corretto utilizzo dell'applicazione devono essere rispettate le seguenti regole di dominio:

ID	Regola	Modificabilità	Sorgente
R1	Aggiungere ingredienti extra (es. Formaggio) aumenta il prezzo base del prodotto.	Bassa, il sovrapprezzo è stabilito dal listino.	Politica interna
R2	Rimuovere ingredienti standard (es. Senza Cipolla) non diminuisce il prezzo base del prodotto.	Bassa.	Politica interna
R3	Se il carrello contiene un set combinato (es. Menu Panino + Patatine + Bibita), il sistema applica automaticamente lo sconto combo definito.	Alta, il manager può decidere di variare la percentuale.	Politica interna

1.7 Specifiche Supplementari

Usabilità

L'interfaccia grafica del chiosco deve essere intuitiva e semplice anche per un utente non esperto. L'interazione con il sistema non deve presentare un elevato grado di complessità. Viene inserito un help in linea con le istruzioni per usare il sistema, anche se deve essere comunque possibile utilizzare il sistema senza la lettura di istruzioni preliminari.

Affidabilità

Il software sviluppato deve essere affidabile e deve poter mantenere i propri dati anche in caso di guasti (guasti elettrici, usura dell'hardware, attacchi informatici). Deve essere possibile pianificare dei backup periodici del database la cui cadenza sarà definita più avanti nel progetto.

Vincoli hardware e software

Per eseguire il software non ci sono particolari requisiti per il sistema operativo purché sia presente la Java Virtual Machine.

Aspetti legali

Le tecnologie utilizzate per la progettazione e realizzazione del sistema proposto sono di tipo open source o freeware.

1.8 Glossario

Vengono qui riportati i termini più significativi e le relative definizioni.

- **Kiosk (Chiosco):** La postazione hardware touch-screen da cui il cliente effettua l'ordinazione.
- **Prodotto:** Elemento base del menu (es. Hamburger, Patatine).
- **Personalizzazione:** Modifica applicata a un prodotto (aggiunta o rimozione di un ingrediente, modifica taglia).
- **Ordine:** Insieme delle voci inserite nel carrello da un cliente e validate tramite pagamento.
- **Voce Ordine:** Singola riga del carrello, indicante prodotto, quantità ed eventuali personalizzazioni.
- **CRUD:** Acronimo di Create, Read, Update, Delete. Indica le operazioni di base per la gestione dei dati (es. Menu o Promozioni).

Capitolo 2

Analisi Orientata agli Oggetti - Iterazione 1

2.1 Introduzione

Conclusa la fase di ideazione, si passa alla fase di elaborazione. Lo scopo di questa prima iterazione è implementare in maniera iterativa il nucleo dell'architettura del software, definendo gli oggetti principali del dominio e le loro responsabilità. Durante questa prima iterazione i requisiti scelti su cui concentrarsi sono:

- Implementare lo scenario principale di successo del caso d'uso **UC1: Gestisci Ordinazione**.
- Tralasciare in questa iterazione, per semplicità, l'associazione delle promozioni all'ordine e la gestione in tempo reale della coda in cucina.
- I dati, per questa fase, verranno gestiti in memoria principale.

2.2 Analisi Orientata agli Oggetti

L'analisi orientata agli oggetti si basa sulla creazione di una descrizione del dominio del problema. Vengono utilizzati tre strumenti principali: il Modello di Dominio, i Diagrammi di Sequenza di Sistema (SSD) e i Contratti delle operazioni.

2.2.1 Modello di Dominio

Relativamente al caso d'uso scelto (UC1), dopo un'attenta valutazione dello scenario principale di successo, è stato possibile identificare le seguenti classi concettuali:

- **StatoOrdine**: contiene l'Enum per rappresentare il cambio di stato dell'ordine.
- **Ordine**: rappresenta l'insieme dei prodotti richiesti dal cliente e il totale da pagare.
- **VoceOrdine**: contiene i dettagli relativi all'aggiunta di un particolare prodotto, la sua quantità e le personalizzazioni.

- **Prodotto:** rappresenta l'articolo base presente nel menu.
- **Personalizzazione:** rappresenta l'aggiunta o rimozione di un ingrediente.
- **Catalogo:** contiene tutte le descrizioni dei prodotti acquistabili.
- **Cliente:** attore primario, che interagisce con il sistema.
- **Chiosco (Sistema):** rappresenta il sistema "Gestore Ordinanze Fast Food".
- **Pagamento:** rappresenta la transazione economica simulata.

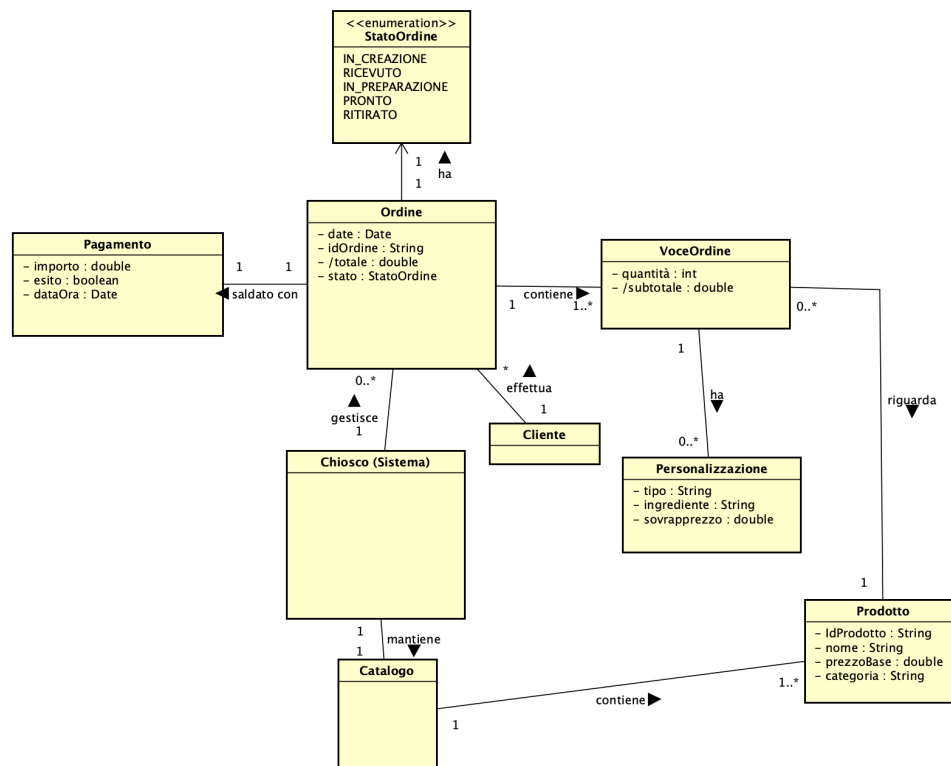


Figura 2.1: Modello di Dominio - Iterazione 1

2.2.2 Diagramma di Sequenza di Sistema (SSD)

Il passo successivo è la creazione del Diagramma di Sequenza di Sistema (SSD) al fine di illustrare il corso degli eventi di input e di output per lo scenario principale di successo di UC1.

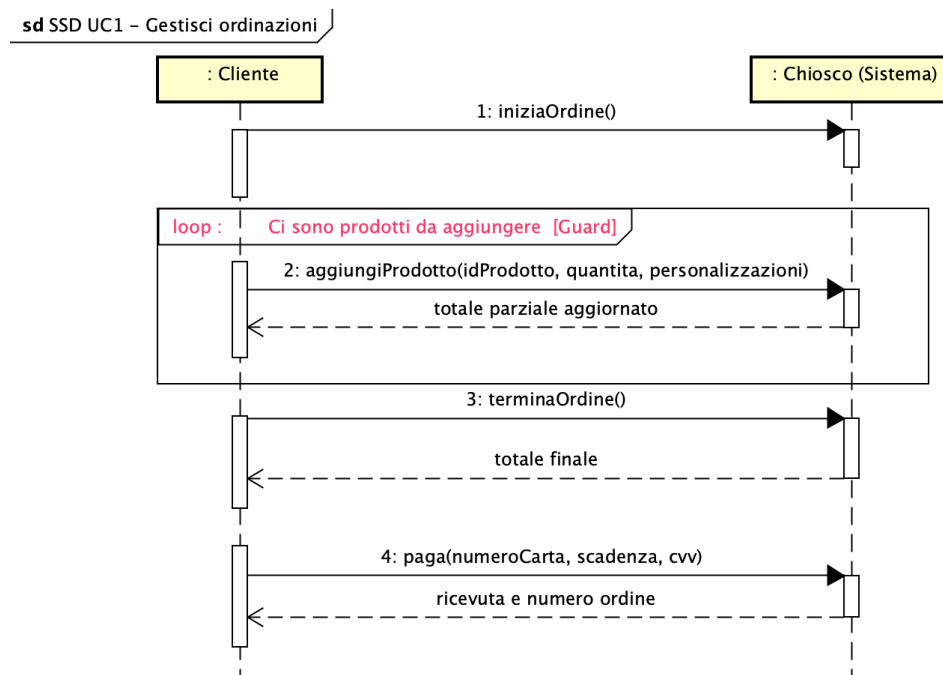


Figura 2.2: SSD per UC1: Gestisci Ordinazione

Gli eventi di sistema individuati nell'SSD per interagire con il *:Sistema* sono:

- `iniziaOrdine()`
- `aggiungiProdotto(idProdotto, quantita, personalizzazioni)`
- `terminaOrdine()`
- `paga(numeroCarta, scadenza, cvv)`

2.2.3 Contratti delle operazioni

Vengono ora descritte attraverso i Contratti le principali operazioni di sistema che si occupano di gestire gli eventi individuati nell'SSD.

Contratto CO1: `iniziaOrdine`

Operazione:	<code>iniziaOrdine()</code>
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	Il chiosco è in attesa di un nuovo cliente.
Post-Condizioni:	- È stata creata una nuova istanza <i>o</i> di Ordine. - L'istanza <i>o</i> è stata associata al Sistema Chiosco come ordine corrente.

Contratto CO2: aggiungiProdotto

Operazione:	aggiungiProdotto(idProdotto: String, quantita: int, personalizzazioni: List<String[]>)
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	È in corso un ordine corrente.
Post-Condizioni:	- È stata creata una nuova istanza <i>vo</i> di VoceOrdine. - Gli attributi di <i>vo</i> sono stati aggiornati con la quantità. - <i>vo</i> è stata associata all'Ordine corrente tramite associazione "contiene". - <i>vo</i> è stata associata al Prodotto corrispondente a <i>idProdotto</i>. - Sono state create eventuali istanze di <i>Personalizzazione</i> associate a <i>vo</i>.

Contratto CO3: terminaOrdine

Operazione:	terminaOrdine()
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	È in corso un ordine con almeno una VoceOrdine.
Post-Condizioni:	- L'attributo totale dell'Ordine corrente è stato calcolato e aggiornato.

Contratto CO4: paga

Operazione:	paga(numeroCarta: String, scadenza: String, cvv: String)
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	L'ordine è terminato e il totale è stato calcolato.
Post-Condizioni:	- È stata creata un'istanza <i>p</i> di Pagamento. - <i>p</i> è stata associata all'Ordine corrente. - L'attributo stato dell'Ordine è stato aggiornato al valore RICEVUTO dell'enum StatoOrdine. - L'Ordine è stato salvato nello storico del Sistema.

2.3 Progettazione Orientata agli Oggetti

La progettazione orientata agli oggetti definisce le responsabilità degli oggetti software e come questi collaborano. In questa fase applichiamo i pattern **GRASP** (come Controller, Creator, Information Expert) per allocare correttamente i metodi alle classi.

2.3.1 Diagrammi di Interazione (Sequenza)

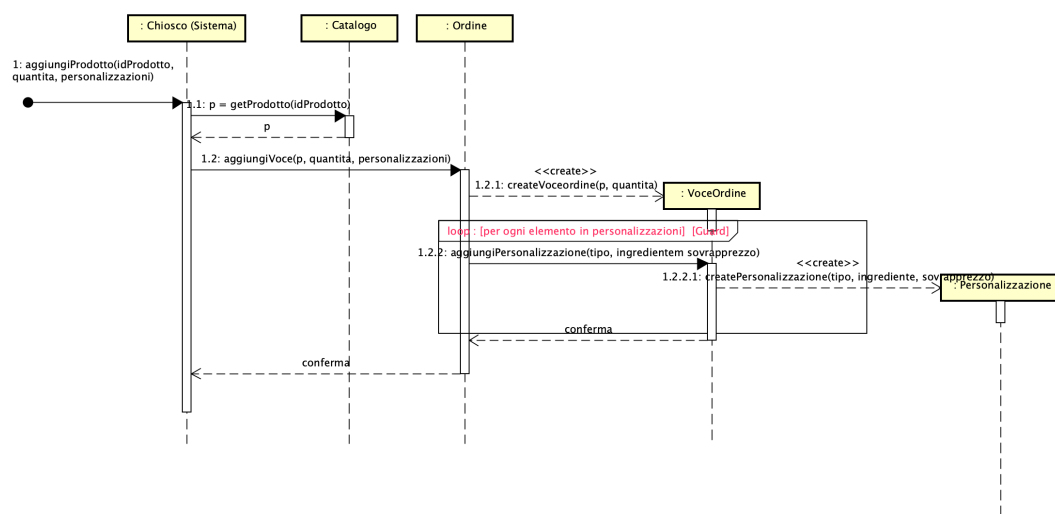


Figura 2.3: Diagramma di Sequenza: aggiungiProdotto

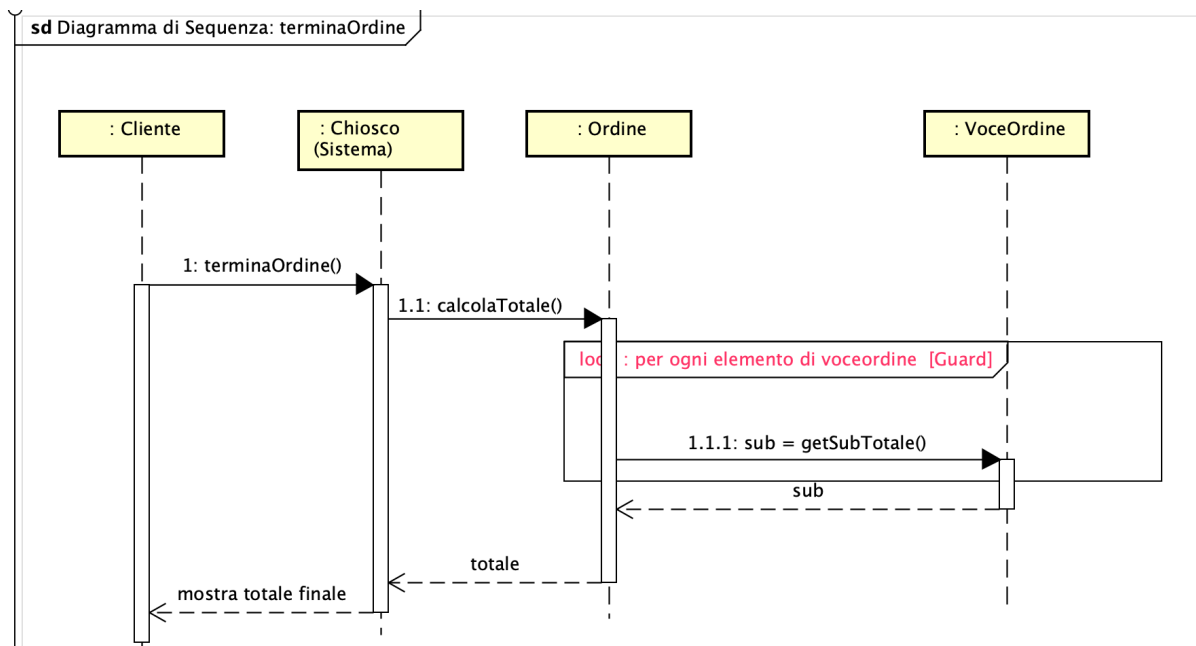


Figura 2.4: Diagramma di Sequenza: terminaOrdine

2.3.2 Diagramma delle Classi di Progetto (DCD)

Il Diagramma delle Classi di Progetto (DCD) evolve dal Modello di Dominio, integrando i metodi ricavati dai diagrammi di sequenza, la visibilità degli attributi e le relazioni di navigabilità.

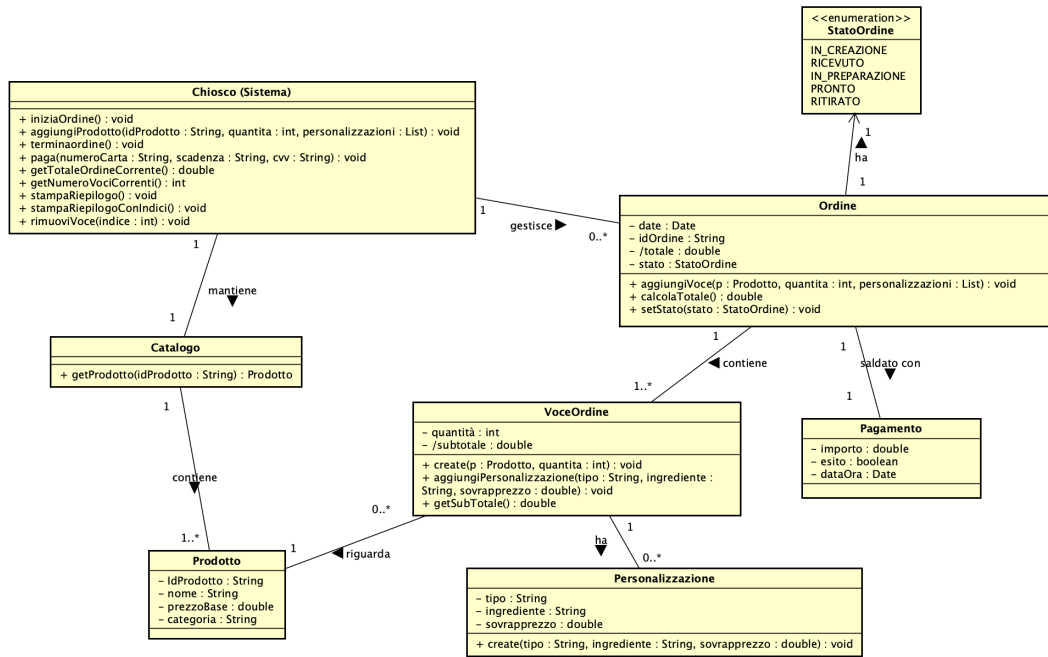


Figura 2.5: Design Class Diagram (DCD) - Iterazione 1

Capitolo 3

Analisi Orientata agli Oggetti - Iterazione 2

3.1 Introduzione

Conclusa con successo la prima iterazione, l'Iterazione 2 si pone l'obiettivo di espandere le funzionalità del sistema coprendo i requisiti precedentemente tralasciati. Durante questa iterazione i requisiti scelti su cui concentrarsi sono:

- Implementare il caso d'uso **UC2: Gestisci Coda Cucina**, permettendo al personale di visualizzare gli ordini e aggiornarne lo stato di preparazione in tempo reale.
- Integrare il sistema di calcolo del chiosco (**UC1**) con l'applicazione automatica delle Promozioni, rispettando le regole di business definite (come l'applicazione automatica degli sconti per i set combinati).

3.2 Analisi Orientata agli Oggetti

Per supportare le nuove funzionalità, il Modello di Dominio viene espanso includendo le entità necessarie per la gestione della cucina e degli sconti.

3.2.1 Modello di Dominio Aggiornato

Oltre alle classi identificate nell'Iterazione 1, sono state introdotte le seguenti classi concettuali:

- **Promozione:** rappresenta una regola di sconto applicabile all'ordine.
- **Personale Cucina:** l'attore primario che interagisce con il terminale di back-office.
- **CodaCucina:** gestisce l'elenco cronologico degli ordini in attesa di preparazione.
- **MonitorSala:** rappresenta il display che notifica ai clienti lo stato del proprio ordine.

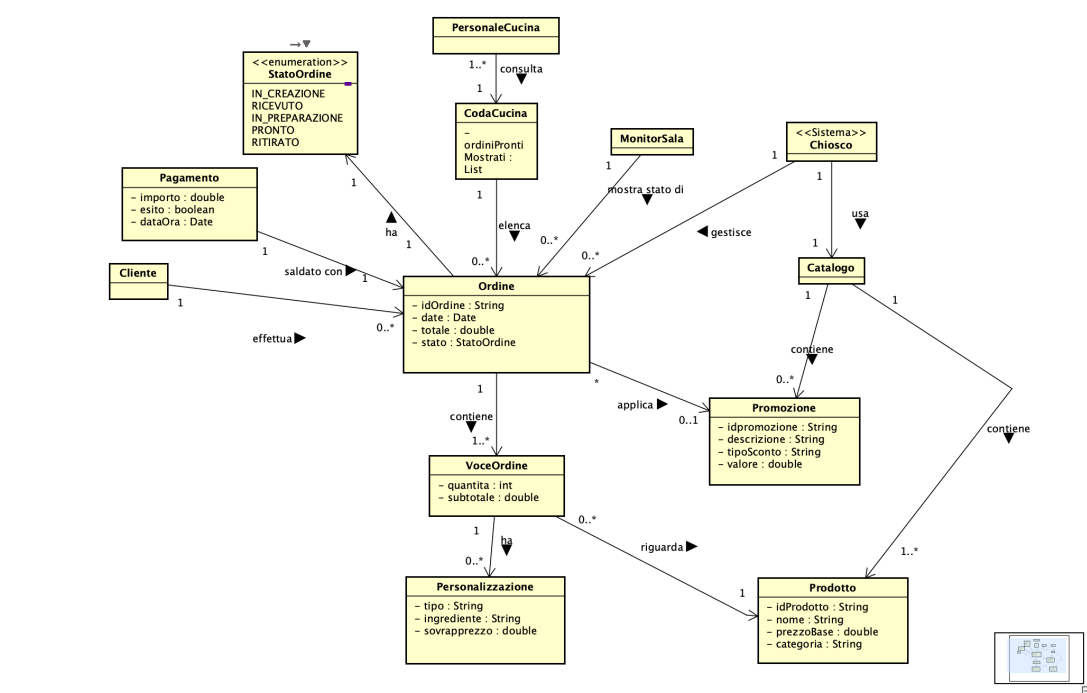


Figura 3.1: Modello di Dominio Aggiornato - Iterazione 2

3.2.2 Diagramma di Sequenza di Sistema (SSD)

Di seguito viene illustrato il Diagramma di Sequenza di Sistema per lo scenario principale di **UC2: Gestisci Coda Cucina**.

sd Diagramma di sequenza di sistema (ssd) iterazione 2

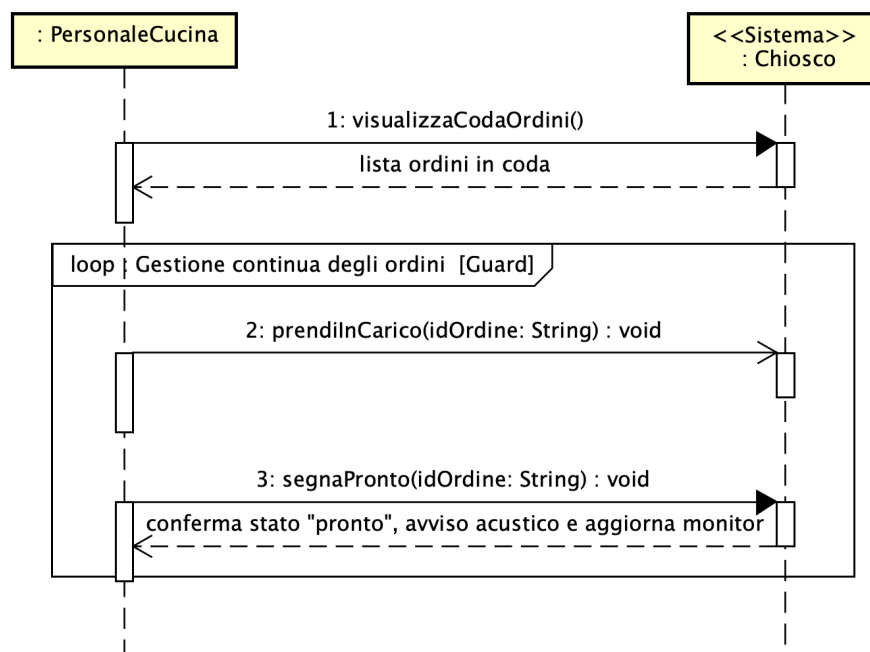


Figura 3.2: SSD per UC2: Gestisci Coda Cucina

Gli eventi di sistema individuati per l'interazione del personale di cucina con il terminale sono:

- `visualizzaCodaOrdini()`
- `prendiInCarico(idOrdine: String)`
- `segnaPronto(idOrdine: String)`

3.2.3 Contratti delle operazioni

Vengono definiti i contratti per le nuove operazioni del sistema.

Contratto CO5: `prendiInCarico`

Operazione:	<code>prendiInCarico(idOrdine: String)</code>
Riferimenti:	Caso d'uso: Gestisci Coda Cucina
Pre-condizioni:	L'ordine corrispondente a <i>idOrdine</i> è in stato <code>RICEVUTO</code> .
Post-Condizioni:	- L'attributo stato dell'Ordine è stato aggiornato al valore <code>IN_PREPARAZIONE</code> . - Il <code>MonitorSala</code> è stato aggiornato per riflettere il nuovo stato.

Contratto CO6: `segnaPronto`

Operazione:	<code>segnaPronto(idOrdine: String)</code>
Riferimenti:	Caso d'uso: Gestisci Coda Cucina
Pre-condizioni:	L'ordine corrispondente a <i>idOrdine</i> è in stato <code>IN_PREPARAZIONE</code> .
Post-Condizioni:	- L'attributo stato dell'Ordine è stato aggiornato al valore <code>PRONTO</code> . - Il <code>MonitorSala</code> è stato aggiornato ed è stato emesso un avviso acustico in sala.

3.3 Progettazione Orientata agli Oggetti

Durante la progettazione dell'Iterazione 2, applichiamo i pattern GRASP per distribuire le nuove responsabilità, delegando all'oggetto `Ordine` il compito di richiedere al Catalogo le Promozioni attive.

3.3.1 Diagrammi di Interazione (Sequenza)

sd Diagramma di Sequenza prendiInCarico

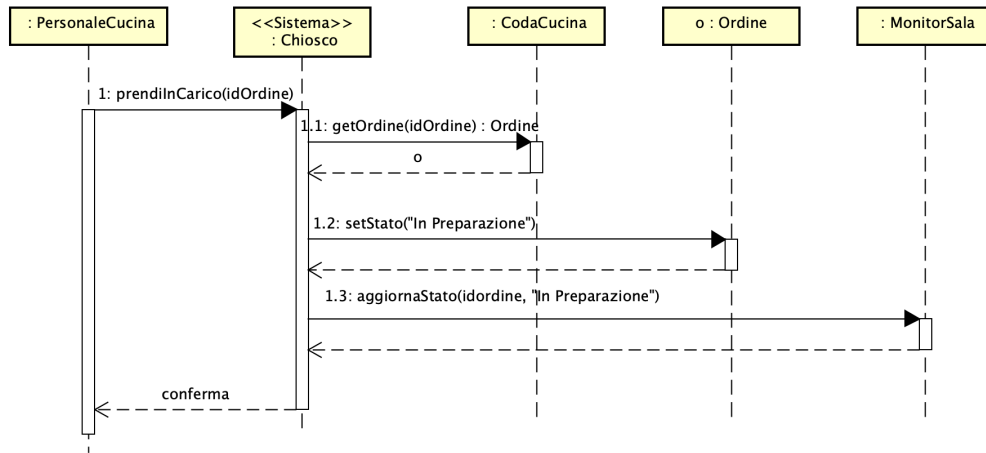


Figura 3.3: Diagramma di Sequenza: prendiInCarico

3.3.2 Diagramma delle Classi di Progetto (DCD) Aggiornato

Il DCD evolve integrando i metodi per la gestione degli stati dell'ordine e le logiche di sconto.

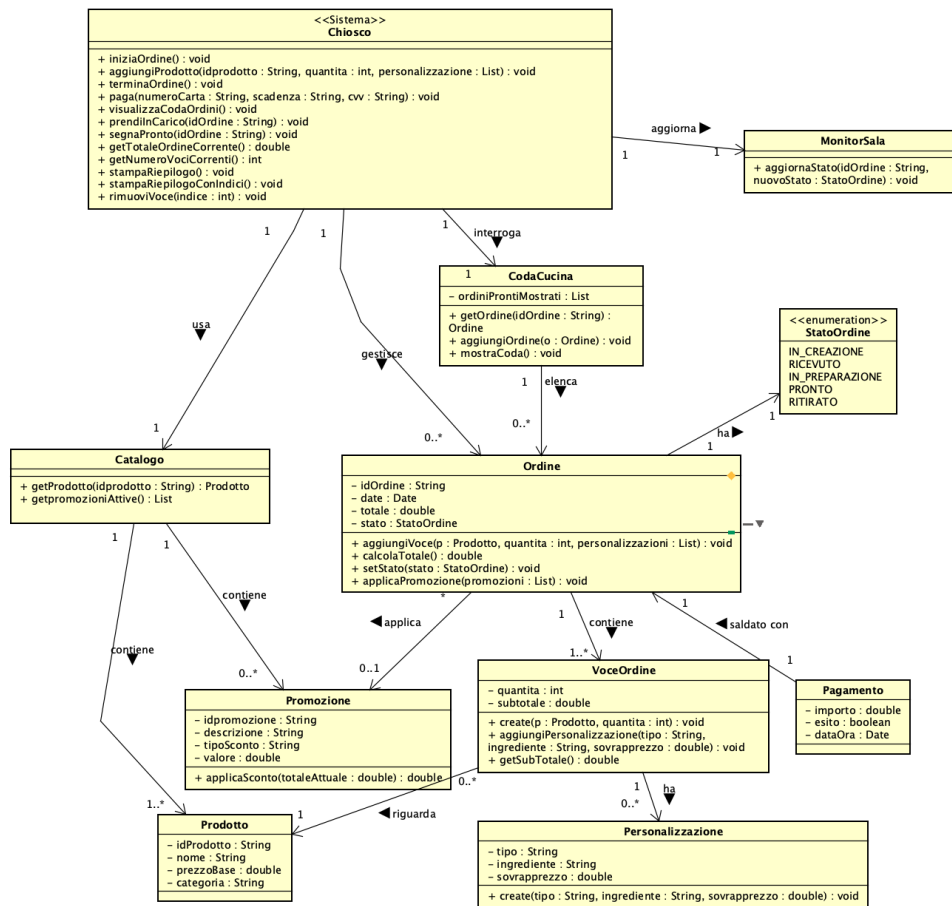


Figura 3.4: Design Class Diagram (DCD) - Iterazione 2

Capitolo 4

Analisi Orientata agli Oggetti - Iterazione 3

4.1 Introduzione

L'Iterazione 3 si pone l'obiettivo di espandere le funzionalità del sistema includendo gli strumenti di reportistica. Durante questa iterazione il requisito scelto su cui concentrarsi è:

- Implementare il caso d'uso **UC5: Genera Report Vendite**, permettendo al manager di consultare le statistiche relative agli incassi e agli ordini completati.

4.2 Analisi Orientata agli Oggetti

4.2.1 Modello di Dominio Aggiornato

Per la generazione del report non è stato necessario introdurre nuove classi concettuali. Il sistema sfrutta lo storico della classe `Ordine` e l'attributo relativo all'importo del `Pagamento` già definiti nelle iterazioni precedenti. Il Modello di Dominio rimane pertanto invariato rispetto all'Iterazione 2.

4.2.2 Diagramma di Sequenza di Sistema (SSD)

Di seguito viene illustrato il Diagramma di Sequenza di Sistema per lo scenario principale di **UC5: Genera Report Vendite**.

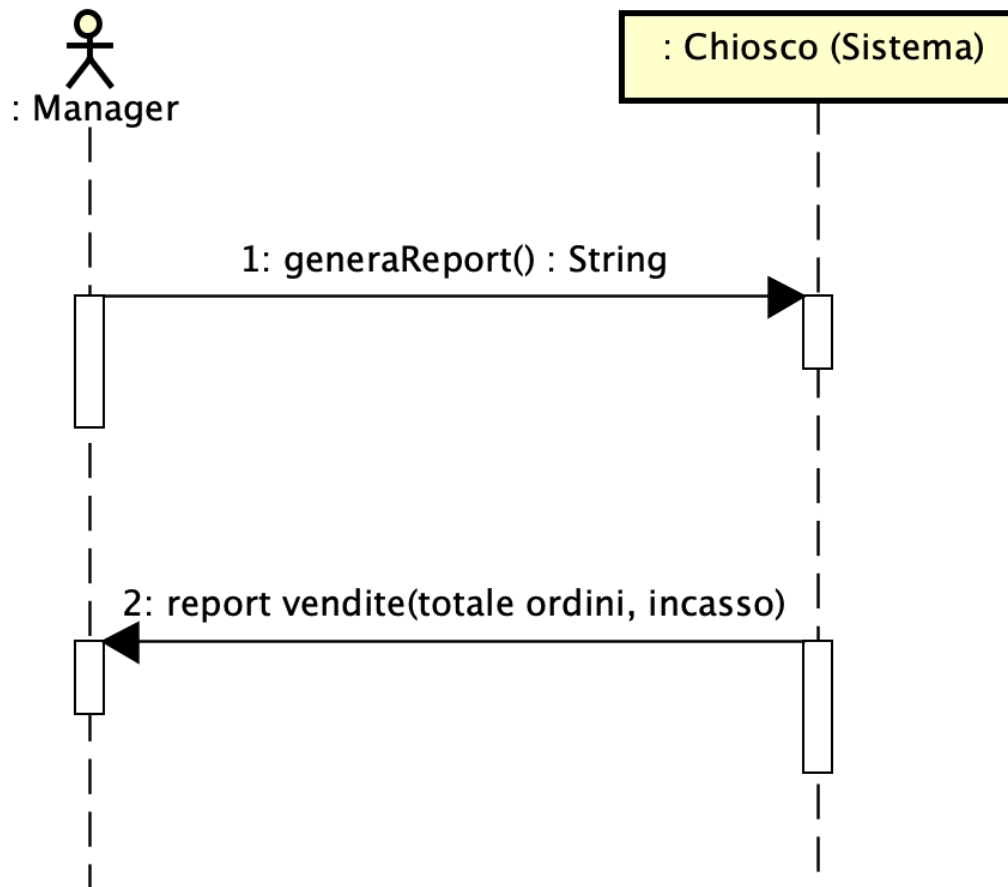


Figura 4.1: SSD per UC5: Genera Report Vendite

Gli eventi di sistema individuati per l'interazione del manager con il terminale sono:

- richiediReport()

4.2.3 Contratti delle operazioni

Viene definito il contratto per la nuova operazione del sistema.

Contratto CO7: richiediReport

Operazione:	richiediReport()
Riferimenti:	Caso d'uso: Genera Report Vendite
Pre-condizioni:	È presente uno storico degli ordini nel sistema.
Post-Condizioni:	- È stato calcolato il conteggio degli ordini evasi. - È stata calcolata la somma totale degli importi (incasso). - I dati sono stati aggregati in un report per la visualizzazione.

4.3 Progettazione Orientata agli Oggetti

Durante la progettazione dell'Iterazione 3, la responsabilità di calcolare il report viene assegnata al Controller, che interroga lo storico degli ordini in modo centralizzato.

4.3.1 Diagrammi di Interazione (Sequenza)

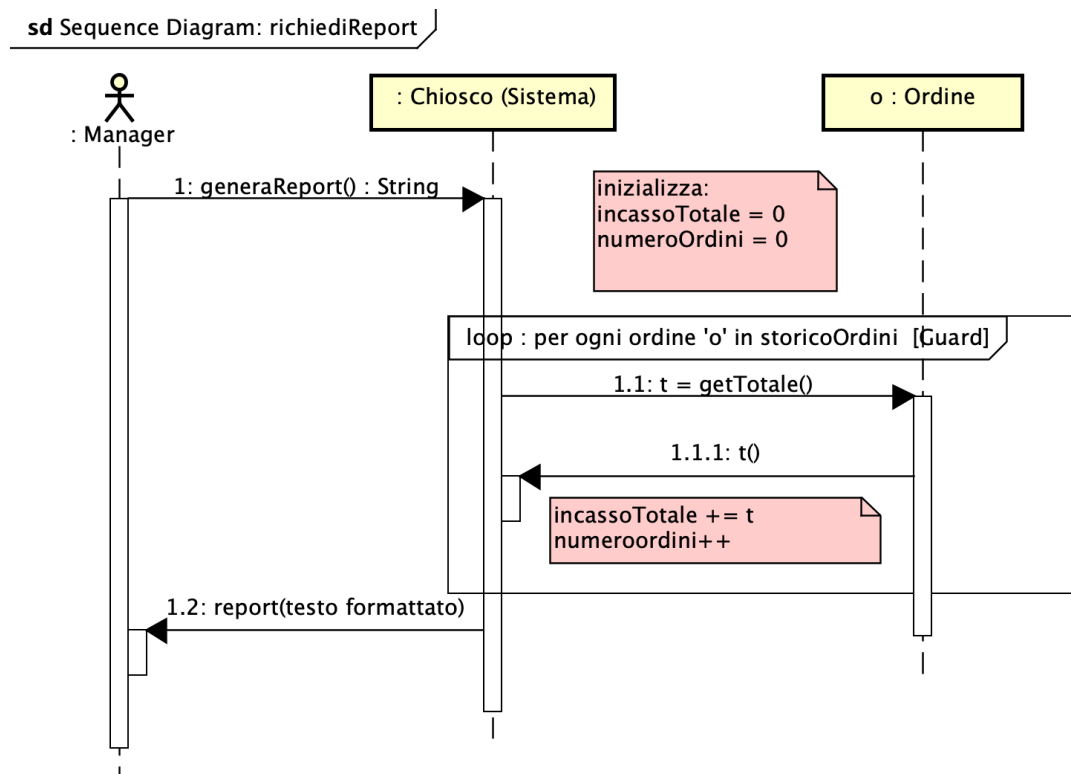


Figura 4.2: Diagramma di Sequenza: richiediReport

Il DCD evolve integrando un array/lista `storicoOrdini` e il metodo `generaReport()` all'interno del Controller `Chiosco`.



Capitolo 5

Testing

5.1 Introduzione

Il testing è una fase essenziale del processo di sviluppo di un programma robusto ed efficiente. Permette inoltre di ridurre in modo sostanziale i costi di manutenzione di un'applicazione. Per eseguire il collaudo del "Gestore di Ordinazioni per Fast Food", si è scelto di combinare i test di unità (condotti con l'ausilio del framework JUnit) ai test di sistema funzionali (eseguiti manualmente sull'applicativo).

5.2 Individuazione dei casi di test e Testing Unitario (JUnit)

In una fase preliminare al testing, è stata affrontata una discussione per operare una scelta sulle classi e sui metodi da testare. Il criterio scelto è stato quello di dare priorità alle logiche di calcolo del dominio e alla correttezza delle transizioni di stato. La strategia di identificazione dei test si è basata sul testing funzionale (Black Box), individuando le classi di equivalenza per le funzioni critiche. In particolare, è stata testata accuratamente l'interazione coordinata dal controller `Chiosco` e la corretta applicazione delle regole di business da parte delle entità di dominio (come `Ordine`, `VoceOrdine` e `Catalogo`).

I casi di test automatizzati validano le seguenti aree funzionali, mappate sulle rispettive classi di test:

- **Gestione Carrello** (`GestioneCarrelloTest.java`): Verifica delle dinamiche di base quali l'aggiunta di prodotti, la rimozione di voci e la modifica delle quantità, assicurando il ricalcolo matematicamente esatto del totale dell'ordine.
- **Transizioni di Stato** (`GestioneStatiOrdineTest.java`): Validazione del corretto avanzamento del ciclo di vita dell'ordine (passaggio da *Ricevuto* a *In Preparazione* fino a *Pronto*) simulando le interazioni del personale di cucina.
- **Validazione Pagamenti** (`PagamentoTest.java`): Controllo del sistema di pagamento simulato, garantendo l'autorizzazione con dati validi e il rifiuto in caso di formati errati o stringhe non conformi.

- **Reportistica Manager** (`StatisticheVenditeTest.java`): Verifica della corretta aggregazione dei dati (numero di ordini evasi e incasso totale netto) a seguito del completamento di molteplici ordini distinti.
- **Personalizzazioni e Subtotali** (`PersonalizzazioneTest.java`): Controllo accurato dei subtotali per le singole voci in caso di aggiunta di ingredienti extra (con sovrapprezzo), rimozioni (sovrapprezzo nullo) o upgrade (es. varianti per i Menu Combo).
- **Robustezza del Sistema** (`CatalogoTest.java` e `CodaCucinaTest.java`): Test sui casi limite per verificare che l'interrogazione di risorse inesistenti (ID prodotto nel catalogo o ID ordine nella coda cucina) sia gestita in sicurezza, restituendo `null` ed evitando eccezioni non gestite a runtime.

L'esecuzione della test suite ha dato esito pienamente positivo (con una percentuale di superamento del 100%). Questo conferma la solidità della logica di business e il rigoroso rispetto dei contratti delle operazioni.

5.3 Test di Sistema

Parallelamente ai test unitari, sono stati eseguiti dei test di sistema manuali lanciando l'interfaccia utente (Swing) e provando a portare a termine i principali casi d'uso (con i rispettivi scenari alternativi) per verificare l'integrazione end-to-end.

Durante il collaudo, l'interfaccia `ClientPanel` (lato chiosco) ha gestito in maniera fluida le operazioni di composizione del carrello, la navigazione a schede e l'inserimento touch dei dati per il pagamento. Allo stesso modo, l'interfaccia `KitchenPanel` (lato cucina) ha ricevuto istantaneamente gli aggiornamenti di stato grazie all'uso di callback implementate nel codice, garantendo la consistenza dei dati tra client e cucina e simulando fedelmente le interazioni del fast food.

Capitolo 6

Architettura, Refactoring e Conclusioni

6.1 Architettura Model-View-Controller (MVC) e Struttura del Codice

L'intero progetto adotta un'architettura software robusta basata sul pattern **Model-View-Controller (MVC)**, affiancato dai pattern GRASP per una corretta assegnazione delle responsabilità. Tale scelta ha permesso di separare nettamente l'applicazione in tre macro-componenti interdipendenti ma ben distinti:

- **Model (Modello):** Costituito dal package `domain`, contiene la logica dei dati e le entità di business (ordini, prodotti, pagamenti, promozioni e stati) indipendentemente dall'interfaccia grafica.
- **View (Vista):** Costituito dal package `main`, gestisce l'interfaccia grafica utente (GUI) implementata in Java Swing. Raccoglie gli input degli attori coinvolti e formatta a video i risultati ricevuti dal Controller senza mai manipolare direttamente la logica applicativa.
- **Controller (Controller e Servizi):** Costituito dalla classe principale `Chiosco` (nel package `controller`) e supportato dal package `services`. Il controller fa da intermediario esclusivo tra la Vista e il Modello, garantendo il corretto flusso esecutivo.

Organizzazione dei File Sorgente (Package)

Il codice sorgente è organizzato rigorosamente in cartelle per riflettere in modo speculare l'architettura MVC sopra descritta:

Cartella `src/controller`

- `Chiosco.java`: Funge da Controller principale (GRASP) e coordina i flussi di business tra carrello, pagamenti, promozioni e code.

Cartella `src/domain` (Model)

- `Catalogo.java`: Gestisce l'anagrafica dei prodotti disponibili e le promozioni.
- `Ordine.java`: Traccia il ciclo di vita dell'ordine, le voci contenute, il totale e il pagamento.
- `Pagamento.java`: Registra i dati e l'esito della transazione economica di un ordine.
- `Personalizzazione.java`: Definisce le variazioni (aggiunte, rimozioni, upgrade) e i relativi sovrapprezzi sui prodotti.
- `Prodotto.java`: Rappresenta un singolo elemento in vendita con i dettagli anagrafici e il prezzo base.
- `Promozione.java`: Gestisce gli sconti applicabili a un ordine di tipo percentuale o fisso.
- `StatoOrdine.java`: Enum che definisce in modo sicuro gli stati dell'ordine (es. In Creazione, Ricevuto, In Preparazione, Pronto, Ritirato).
- `VoceOrdine.java`: Rappresenta una riga del carrello associata a un prodotto, alla quantità e alle personalizzazioni.

Cartella `src/main` (View)

- `BadgePanel.java`: Disegna un indicatore grafico a cerchio per le notifiche o le quantità.
- `ClientPanel.java`: Gestisce l'interfaccia cliente per la scelta dei prodotti, la gestione del carrello, la modifica degli ingredienti e il pagamento.
- `KitchenPanel.java`: Gestisce l'interfaccia del personale di cucina e il monitor sala, con comandi rapidi e avvisi acustici.
- `MainGUI.java`: Classe principale di avvio che assembla il controller e le viste all'interno di un frame a schede in modo thread-safe.
- `ManagerPanel.java`: Interfaccia di back-office per la visualizzazione del report delle vendite giornaliere.
- `RoundButton.java`: Componente grafico personalizzato per bottoni con angoli arrotondati.
- `RoundedPanel.java`: Componente grafico personalizzato per pannelli con angoli smussati.

Cartella `src/services`

- `Catalogo.java` (noto internamente come `Catalogo_2.java`): Gestisce listini predefiniti e cataloghi di supporto ai servizi.
- `CodaCucina.java`: Gestisce la lista degli ordini in coda di elaborazione per la cucina e formatta la visualizzazione per il personale.

- `MonitorSala.java`: Gestisce lo stato pubblico degli ordini da mostrare ai clienti, dividendoli tra in preparazione e pronti per il ritiro.

Cartella test (Suite JUnit)

- `GestioneCarrelloTest.java`: Testa le dinamiche del carrello virtuale (aggiunta, rimozione e modifica quantità delle voci).
- `GestioneStatiOrdineTest.java`: Verifica la transizione corretta degli stati dell'ordine (da Ricevuto a In Preparazione fino a Pronto).
- `PagamentoTest.java`: Valida il sistema di pagamento simulato con dati corretti o errati.
- `StatisticheVenditeTest.java`: Verifica la correttezza del report vendite giornaliero simulando più ordini completati.
- `PersonalizzazioneTest.java`: Verifica il calcolo esatto dei subtotali per le singole voci a seguito di aggiunte extra, rimozioni o upgrade.
- `CatalogoTest.java`: Testa la robustezza della ricerca dei prodotti validando input corretti, categorie e la gestione di ID inesistenti.
- `CodaCucinaTest.java`: Verifica la stabilità della coda ordini e del sistema quando vengono richiesti ID inesistenti o non validi.

6.2 Refactoring

Nella fase finale del progetto, in seguito ai test di sistema, è stata effettuata una revisione del codice (refactoring) per migliorare la qualità architetturale e pulire la base di codice. In particolare, le viste grafiche (le classi interne al *main*) sono state disaccoppiate in maniera più netta dalla logica di business, in piena aderenza al pattern architetturale MVC descritto in precedenza. I flussi dell'ordine, l'accesso al `Catalogo` e l'aggiornamento della `CodaCucina` sono stati incapsulati esclusivamente nel controller `Chiosco`, rispettando fedelmente i pattern architetturali GRASP trattati nella fase di progettazione.

In accordo con le regole di progetto, si è optato per non implementare un database relazionale; i dati rimangono pertanto gestiti in memoria in maniera efficiente tramite apposite classi mock nel layer dei servizi.

6.3 Test di Accettazione e Conclusioni

A conclusione delle attività necessarie per portare a compimento i passi precedenti, e dopo un'attenta revisione di tutto il codice sorgente, è stato eseguito il test di accettazione conclusivo.

Simulando l'interazione sia dell'utente finale (cliente al chiosco) sia dello staff (personale in cucina al back-office), è stato dimostrato che il sistema è robusto e funzionale. Il software sviluppato permette di portare a termine con successo l'intero ciclo di vita di un'ordinazione, dalla scelta dei prodotti e relative personalizzazioni all'emissione della

ricevuta e alla corretta gestione dell'ordine in cucina, soddisfacendo appieno i requisiti individuati nella fase iniziale di ideazione.